Relatório de Projeto: Compilador SL
Etapa 2 – Análise Semântica e Interpretador

Kayo Xavier – Matrícula: 21.2.4095
Pedro Vieira – Matrícula: 20.2.4008

BCC328 – Construção de Compiladores I – DECOM/UFOP

21 de fevereiro de 2026

Resumo

Este relatório descreve a segunda etapa do desenvolvimento do compilador para a linguagem SL (*Simple Language*), implementado em Haskell. Nesta etapa, foram projetados e implementados o analisador semântico e o interpretador da linguagem. O analisador semântico realiza verificações de tipos, resolução de nomes e validação de escopos, enquanto o interpretador executa programas SL por meio de caminhamento na árvore de sintaxe abstrata (AST). O documento apresenta o sistema de tipos formalizado, a semântica operacional de *big-step*, as decisões de projeto e os resultados obtidos nos testes com os programas de exemplo da especificação.

Sumário

1 Introdução **3**

2 Metodologia **3**
2.1 Estrutura sintática de SL . . . 3
2.2 Sistema de tipos para SL . . . 3
2.2.1 Tipos da linguagem . . . 4
2.2.2 Compatibilidade de tipos . . . 4
2.2.3 Regras de tipagem . . . 4
2.3 Inferência de tipos para SL . . . 5
2.4 Semântica operacional para SL . . . 6
2.4.1 Valores de tempo de execução . . . 6
2.4.2 Regras para expressões . . . 6
2.4.3 Regras para instruções . . . 6

3 Arquitetura do Compilador **7**
3.1 Análise léxica . . . 7
3.2 Análise sintática . . . 8
3.3 Árvore de sintaxe abstrata . . . 8
3.4 Análise semântica . . . 8
3.5 Interpretador . . . 9

4 Resultados e Discussão **10**
4.1 Instruções de Uso 10
4.2 Testes Realizados 11
4.3 Limitações 12

5 Conclusão **12**

6 Referências **12**

1 Introdução

A linguagem SL (*Simple Language*) é uma linguagem de programação imperativa com tipagem estática, projetada para fins didáticos no contexto da disciplina BCC328 – Construção de Compiladores I. A linguagem possui suporte a tipos primitivos (`int`, `float`, `string`, `bool`), estruturas de controle (`if-else`, `while`, `for`), arranjos unidimensionais, registros (*structs*), funções com parâmetros e retorno, polimorfismo paramétrico (`forall`) e inferência de tipos.

Na primeira etapa do trabalho, foram implementados o analisador léxico e o analisador sintático utilizando a biblioteca Megaparsec, resultando na construção da árvore de sintaxe abstrata (AST) a partir do código-fonte SL.

Nesta segunda etapa, o objetivo foi implementar duas componentes fundamentais:

1. **Analisador Semântico**: responsável por verificar a correção semântica dos programas, incluindo compatibilidade de tipos, unicidade de declarações, verificação de chamadas de função e acesso válido a campos de registros e elementos de arranjos.

2. **Interpretador**: responsável por executar programas SL diretamente a partir da AST, implementando a semântica operacional da linguagem.

O compilador foi desenvolvido inteiramente em Haskell, utilizando o sistema de build Cabal, e está preparado para execução em ambiente Docker conforme especificado pelo professor.

2 Metodologia

2.1 Estrutura sintática de SL

A gramática de SL, definida na Etapa 1, segue uma estrutura hierárquica composta por declarações de topo (*top-level*), que podem ser definições de funções ou de registros. Cada função possui um nome, uma lista de parâmetros (opcionalmente tipados), um tipo de retorno opcional e um corpo composto por instruções (*statements*). A gramática é processada por um analisador sintático descendente recursivo implementado com Megaparsec e a biblioteca `parser-combinators` para tratamento de precedência de operadores.

A AST resultante é definida no módulo `SL.AST` com os seguintes tipos principais:

- `Program`: lista de declarações de topo (`TopLevel`)
- `TopLevel`: `TLFunc` (funções) ou `TLStruct` (registros)
- `Stmt`: instruções como `SLet`, `SAssign`, `SIf`, `SWhile`, `SFor`, `SReturn`, `SBlock`
- `Expr`: expressões como literais, variáveis, operações binárias/unárias, chamadas de função, acesso a arranjos/campos, construtores de arranjo e registro
- `Type`: tipos como `TyInt`, `TyFloat`, `TyString`, `TyBool`, `TyVoid`, `TyVar` (variável de tipo), `TyArray`, `TyStruct`, `TyFunc`

2.2 Sistema de tipos para SL

O sistema de tipos de SL é estático e nominalmente tipado. Definimos formalmente os tipos da linguagem e as regras de tipagem utilizadas pelo analisador semântico.

2.2.1 Tipos da linguagem

Os tipos de SL são definidos pela seguinte gramática:

$$\tau ::= \texttt{int} \mid \texttt{float} \mid \texttt{string} \mid \texttt{bool} \mid \texttt{void} \mid \alpha \mid \tau[\,] \mid S \mid (\tau_1, \ldots, \tau_n) \to \tau$$

onde α denota uma variável de tipo (utilizada em funções genéricas), $\tau[\,]$ denota um arranjo de elementos do tipo τ, S denota um tipo de registro (struct) definido pelo usuário, e $(\tau_1, \ldots, \tau_n) \to \tau$ denota um tipo funcional.

2.2.2 Compatibilidade de tipos

Dois tipos τ_1 e τ_2 são considerados compatíveis ($\tau_1 \sim \tau_2$) se satisfizerem uma das seguintes condições:

1. $\tau_1 = \tau_2$ (igualdade estrutural)
2. τ_1 ou τ_2 é uma variável de tipo α (unificação com qualquer tipo)
3. $\tau_1, \tau_2 \in \{\texttt{int}, \texttt{float}\}$ (coerção numérica implícita)
4. $\tau_1 = \tau_1'[\,]$ e $\tau_2 = \tau_2'[\,]$ com $\tau_1' \sim \tau_2'$ (compatibilidade de arranjos)
5. $\tau_1 = (\sigma_1, \ldots, \sigma_n) \to \rho_1$ e $\tau_2 = (\sigma_1', \ldots, \sigma_n') \to \rho_2$ com $\sigma_i \sim \sigma_i'$ para todo i e $\rho_1 \sim \rho_2$ (compatibilidade de tipos funcionais)

2.2.3 Regras de tipagem

As principais regras de tipagem são apresentadas a seguir, usando a notação $\Gamma \vdash e : \tau$ para indicar que no ambiente Γ, a expressão e possui tipo τ.

Literais.

$$\frac{}{\Gamma \vdash n : \texttt{int}} \qquad \frac{}{\Gamma \vdash f : \texttt{float}} \qquad \frac{}{\Gamma \vdash s : \texttt{string}} \qquad \frac{}{\Gamma \vdash b : \texttt{bool}}$$

Variáveis.

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau}$$

Operações aritméticas.

$$\frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma \vdash e_2 : \tau_2 \qquad \tau_1, \tau_2 \in \{\texttt{int}, \texttt{float}\} \qquad \oplus \in \{+, -, *, /, \%\}}{\Gamma \vdash e_1 \oplus e_2 : \text{wider}(\tau_1, \tau_2)}$$

onde $\text{wider}(\tau_1, \tau_2) = \texttt{float}$ se $\tau_1 = \texttt{float}$ ou $\tau_2 = \texttt{float}$; caso contrário, $\text{wider}(\tau_1, \tau_2) = \texttt{int}$.

Operações de comparação.

$$\frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma \vdash e_2 : \tau_2 \qquad \tau_1, \tau_2 \in \{\texttt{int}, \texttt{float}\} \qquad \oplus \in \{<, \leq, >, \geq\}}{\Gamma \vdash e_1 \oplus e_2 : \texttt{bool}}$$

Operações de igualdade.

$$\frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma \vdash e_2 : \tau_2 \qquad \tau_1 \sim \tau_2 \qquad \oplus \in \{==, !=\}}{\Gamma \vdash e_1 \oplus e_2 : \texttt{bool}}$$

Operações lógicas.

$$\frac{\Gamma \vdash e_1 : \texttt{bool} \qquad \Gamma \vdash e_2 : \texttt{bool} \qquad \oplus \in \{\&\&, ||\}}{\Gamma \vdash e_1 \oplus e_2 : \texttt{bool}}$$

Operações unárias.

$$\frac{\Gamma \vdash e : \tau \qquad \tau \in \{\texttt{int}, \texttt{float}\}}{\Gamma \vdash -e : \tau} \qquad\qquad \frac{\Gamma \vdash e : \texttt{bool}}{\Gamma \vdash !e : \texttt{bool}}$$

Chamada de função.

$$\frac{f : (\tau_1, \ldots, \tau_n) \to \tau_r \in \Gamma \qquad \Gamma \vdash a_i : \sigma_i \qquad \tau_i \sim \sigma_i \text{ para } 1 \leq i \leq n}{\Gamma \vdash f(a_1, \ldots, a_n) : \tau_r}$$

Acesso a arranjo.

$$\frac{\Gamma \vdash e_1 : \tau[\,] \qquad \Gamma \vdash e_2 : \texttt{int}}{\Gamma \vdash e_1[e_2] : \tau}$$

Acesso a campo de registro.

$$\frac{\Gamma \vdash e : S \qquad \text{field}(S, f) = \tau}{\Gamma \vdash e.f : \tau}$$

Propriedade `.size` de arranjos.

$$\frac{\Gamma \vdash e : \tau[\,]}{\Gamma \vdash e.\texttt{size} : \texttt{int}}$$

2.3 Inferência de tipos para SL

A inferência de tipos em SL é utilizada em dois contextos principais:

1. **Variáveis sem anotação de tipo**: quando uma declaração `let x = e;` não possui anotação de tipo, o tipo de `x` é inferido a partir do tipo da expressão `e`.

2. **Parâmetros de função sem anotação**: quando um parâmetro de função não possui anotação de tipo (como em `func id(x) { return x; }`), uma variável de tipo fresca $_t_i$ é gerada e associada ao parâmetro.

A geração de variáveis de tipo frescas é implementada por um contador monotonicamente crescente no ambiente de checagem. Cada nova variável recebe o nome $_t_0, _t_1, \ldots$, garantindo unicidade.

Para funções genéricas declaradas com `forall`, as variáveis de tipo explícitas (e.g., `a`, `b`) são registradas no ambiente como `TyVar`, permitindo que sejam unificadas com qualquer tipo concreto durante a verificação.

2.4 Semântica operacional para SL

A semântica operacional de SL é definida no estilo *big-step* (semântica natural), onde a avaliação de uma expressão ou execução de uma instrução produz diretamente o resultado final. A notação utilizada é:

- $\langle e, \sigma \rangle \Downarrow v$: a expressão e avaliada no estado σ produz o valor v
- $\langle s, \sigma \rangle \Downarrow \sigma'$: a instrução s executada no estado σ produz um novo estado σ'

onde σ é um ambiente de execução que mapeia nomes a valores.

2.4.1 Valores de tempo de execução

Os valores de tempo de execução são definidos por:

$$v ::= n \mid f \mid s \mid b \mid \texttt{void} \mid \langle \texttt{arr}, \text{ref} \rangle \mid \langle \texttt{struct}, S, \text{campos} \rangle \mid \langle \texttt{func}, \text{def} \rangle$$

onde n é um inteiro, f um ponto flutuante, s uma string, b um booleano, $\langle \texttt{arr}, \text{ref} \rangle$ uma referência mutável a um arranjo, e $\langle \texttt{struct}, S, \text{campos} \rangle$ uma referência mutável a um registro.

2.4.2 Regras para expressões

Literais.

$$\frac{}{\langle n, \sigma \rangle \Downarrow n} \qquad \frac{}{\langle f, \sigma \rangle \Downarrow f} \qquad \frac{}{\langle s, \sigma \rangle \Downarrow s} \qquad \frac{}{\langle b, \sigma \rangle \Downarrow b}$$

Variável.

$$\frac{\sigma(x) = v}{\langle x, \sigma \rangle \Downarrow v}$$

Operação binária.

$$\frac{\langle e_1, \sigma \rangle \Downarrow v_1 \qquad \langle e_2, \sigma \rangle \Downarrow v_2 \qquad v_1 \oplus v_2 = v}{\langle e_1 \oplus e_2, \sigma \rangle \Downarrow v}$$

Chamada de função.

$$\frac{f \mapsto (\overline{p_i}, \text{corpo}) \in \sigma \qquad \langle a_i, \sigma \rangle \Downarrow v_i \qquad \sigma' = \sigma[p_i \mapsto v_i] \qquad \langle \text{corpo}, \sigma' \rangle \Downarrow_{\text{ret}} v}{\langle f(\overline{a_i}), \sigma \rangle \Downarrow v}$$

2.4.3 Regras para instruções

Declaração de variável (`let`).

$$\frac{\langle e, \sigma \rangle \Downarrow v}{\langle \texttt{let}\ x\ \texttt{=}\ e, \sigma \rangle \Downarrow \sigma[x \mapsto v]} \qquad \frac{\tau = \text{tipo anotado} \qquad v_0 = \text{default}(\tau)}{\langle \texttt{let}\ x : \tau, \sigma \rangle \Downarrow \sigma[x \mapsto v_0]}$$

Atribuição.

$$\frac{\langle e, \sigma\rangle \Downarrow v}{\langle x \texttt{ = } e, \sigma\rangle \Downarrow \sigma[x \mapsto v]}$$

Condicional (`if-else`).

$$\frac{\langle e, \sigma\rangle \Downarrow \texttt{true} \qquad \langle s_1, \sigma\rangle \Downarrow \sigma'}{\langle \texttt{if } (e)\ s_1 \texttt{ else } s_2, \sigma\rangle \Downarrow \sigma'} \qquad \frac{\langle e, \sigma\rangle \Downarrow \texttt{false} \qquad \langle s_2, \sigma\rangle \Downarrow \sigma'}{\langle \texttt{if } (e)\ s_1 \texttt{ else } s_2, \sigma\rangle \Downarrow \sigma'}$$

Laço `while`.

$$\frac{\langle e, \sigma\rangle \Downarrow \texttt{true} \qquad \langle s, \sigma\rangle \Downarrow \sigma' \qquad \langle \texttt{while } (e)\ s, \sigma'\rangle \Downarrow \sigma''}{\langle \texttt{while } (e)\ s, \sigma\rangle \Downarrow \sigma''} \qquad \frac{\langle e, \sigma\rangle \Downarrow \texttt{false}}{\langle \texttt{while } (e)\ s, \sigma\rangle \Downarrow \sigma}$$

Retorno.

$$\frac{\langle e, \sigma\rangle \Downarrow v}{\langle \texttt{return } e, \sigma\rangle \Downarrow_{\text{ret}} v}$$

3 Arquitetura do Compilador

O compilador SL é organizado em uma arquitetura modular, com cada fase implementada como um módulo Haskell independente. A estrutura segue o padrão clássico de *pipeline* de compilação:

$$\texttt{Código-fonte} \xrightarrow{\text{Lexer}} \texttt{Tokens} \xrightarrow{\text{Parser}} \texttt{AST} \xrightarrow{\text{TypeChecker}} \texttt{AST validada} \xrightarrow{\text{Interpreter}} \texttt{Execução}$$

3.1 Análise léxica

O analisador léxico é implementado no módulo `SL.Lexer` utilizando a biblioteca Megaparsec. O lexer converte o código-fonte em uma sequência de *tokens*, cada um acompanhado de informação de posição (linha e coluna).

Os tokens reconhecidos incluem:

- Literais: inteiros, ponto flutuante, strings e booleanos
- Identificadores e palavras reservadas: `func`, `struct`, `if`, `else`, `while`, `for`, `return`, `let`, `new`, `forall`, e tipos (`int`, `float`, `string`, `bool`, `void`)
- Operadores: `+`, `-`, `*`, `/`, `%`, `==`, `!=`, `<`, `<=`, `>`, `>=`, `&&`, `||`, `!`
- Pontuação: parênteses, chaves, colchetes, ponto-e-vírgula, vírgula, dois-pontos, ponto, seta (`->`)

O lexer também trata comentários de linha (`//`) e de bloco (`/* */`).

3.2 Análise sintática

O analisador sintático é implementado no módulo `SL.Parser`, também utilizando Megaparsec. O parser é um analisador descendente recursivo que constrói a AST diretamente.

A precedência de operadores é tratada pela função `makeExprParser` da biblioteca `parser-combinators`, com a seguinte tabela de precedência (da mais alta para a mais baixa):

1. Operadores unários: `-` (negação), `!` (negação lógica)
2. Multiplicativos: `*`, `/`, `%`
3. Aditivos: `+`, `-`
4. Comparação: `<=`, `>=`, `<`, `>`
5. Igualdade: `==`, `!=`
6. E lógico: `&&`
7. Ou lógico: `||`

Expressões pós-fixas (acesso a arranjo `[i]`, acesso a campo `.field` e chamada de função `(args)`) são tratadas como operações de *postfix* aplicadas ao termo base.

3.3 Árvore de sintaxe abstrata

A AST é definida no módulo `SL.AST` utilizando tipos algébricos de Haskell. As principais decisões de projeto incluem:

- **Tipos opcionais em parâmetros**: `FuncParam` possui `paramType :: Maybe Type`, permitindo que parâmetros sem anotação tenham seu tipo inferido.
- **Tipos de retorno opcionais**: `funcRetType :: Maybe Type` em `TLFunc` permite inferência do tipo de retorno.
- **Variáveis de tipo genérico**: `funcTypeVars :: [TypeVar]` armazena as variáveis de tipo declaradas com `forall`.
- **Tipo funcional**: `TyFunc [Type] Type` representa tipos de funções como `(a) -> b`, suportando funções de alta ordem.
- **Tamanho opcional em arranjos**: `TyArray Type (Maybe Expr)` permite tanto `int[]` quanto `int[5]`.

3.4 Análise semântica

O analisador semântico é implementado nos módulos `SL.TypeChecker` e `SL.Env`. A verificação é realizada em duas passadas sobre a AST:

Passada 1 – Registro de declarações: Todas as funções e structs são registradas no ambiente global antes de verificar os corpos. Isso permite referências mútuas entre funções e o uso de structs antes de suas definições.

Passada 2 – Verificação de corpos: Cada corpo de função é verificado quanto a:

- **Resolução de nomes**: verifica se variáveis e funções referenciadas estão declaradas no escopo visível.
- **Compatibilidade de tipos**: valida operações aritméticas, lógicas, de comparação e de igualdade conforme as regras de tipagem.
- **Declaração única no escopo**: impede a redeclaração de variáveis no mesmo escopo utilizando `lookupVarCurrentScope`.
- **Verificação de chamadas de função**: confere o número e tipos dos argumentos contra a assinatura registrada.
- **Acesso a campos**: valida que acessos `e.field` referem-se a campos existentes do struct correspondente.
- **Indexação de arranjos**: verifica que o operando é um arranjo e que o índice é do tipo `int`.
- **Tipos de retorno**: valida que instruções `return` são compatíveis com o tipo de retorno declarado.

O ambiente de verificação (`SL.Env`) utiliza uma pilha de escopos (lista de `Map Text Type`) para variáveis, com mapas globais para funções e structs. Erros semânticos são acumulados (não interrompem a verificação), permitindo reportar todos os erros de uma só vez.

Suporte a funções de alta ordem. Para chamadas como `f(v[i])` onde `f` é um parâmetro de função com tipo `(a) -> b`, o verificador consulta o ambiente de variáveis caso a função não seja encontrada no registro global, tratando variáveis do tipo `TyFunc` como funções invocáveis.

3.5 Interpretador

O interpretador é implementado no módulo `SL.Interpreter` como um *tree-walking interpreter* que avalia a AST diretamente na mônada `IO` de Haskell. As principais decisões de projeto são:

Valores de tempo de execução. Os valores são representados pelo tipo algébrico `Value`:

```haskell
data Value
    = VInt Integer
    | VFloat Double
    | VString Text
    | VBool Bool
    | VArray (IORef [IORef Value])
    | VStruct Text (IORef (Map Text (IORef Value)))
    | VFunc FuncDef
    | VVoid
```

Mutabilidade via `IORef`. Arranjos e registros são representados usando `IORef` (referências mutáveis), permitindo que atribuições como `arr[i] = v` e `person.name = "Bob"` modifiquem os valores in-place, conforme esperado em uma linguagem imperativa.

Ambiente de execução. O ambiente de tempo de execução utiliza uma pilha de escopos similar ao analisador semântico, porém mapeando nomes a `IORef Value` em vez de tipos:

```
data Env = Env
    { envScopes     :: [Map Text (IORef Value)]
    , envGlobals    :: IORef (Map Text FuncDef)
    , envStructDefs :: Map Text StructDefRT
    }
```

Implementação de `return`. Instruções `return` são implementadas utilizando o mecanismo de exceções de Haskell (`throwIO`/`catch`). Quando um `return e` é executado, uma exceção `ReturnException v` é lançada e capturada pelo `callFunction` que invocou a função, extraindo o valor de retorno.

Inicialização padrão de variáveis. Declarações como `let people : Person[3];` (sem inicializador) criam valores padrão baseados no tipo anotado: `0` para `int`, `0.0` para `float`, para `string`, `false` para `bool`, e arranjos pré-alocados com o tamanho especificado para tipos de arranjo.

Execução em três passadas.

1. **Registro de structs**: todas as definições de struct são coletadas no ambiente.
2. **Registro de funções**: todas as definições de função são registradas como `FuncDef` no mapa global.
3. **Chamada de `main`**: a função `main` é localizada e invocada com argumentos vazios.

Função `print` embutida. A função `print` é tratada como uma função *built-in* que aceita qualquer número de argumentos e imprime cada valor em uma linha separada, convertendo para sua representação textual.

4 Resultados e Discussão

4.1 Instruções de Uso

O compilador pode ser compilado e executado utilizando Cabal dentro do ambiente Docker fornecido:

```
# Compilar o projeto
cabal build

# Executar um programa SL
cabal run sl-compiler -- --interpret programa.sl

# Modos disponiveis
```

```
cabal run sl-compiler -- --lexer programa.sl       # tokens
cabal run sl-compiler -- --parser programa.sl      # AST
cabal run sl-compiler -- --pretty programa.sl      # codigo reconstruido
cabal run sl-compiler -- --typecheck programa.sl   # analise semantica
cabal run sl-compiler -- --interpret programa.sl   # execucao
cabal run sl-compiler -- --run programa.sl         # alias para interpret
```

Para iniciar o ambiente Docker:

```
docker run -it --rm -v ./workspace:/workspace \
    -w /workspace trabalho-02-awsome-team-name-sl bash
```

4.2 Testes Realizados

O projeto conta com 144 testes automatizados utilizando a biblioteca HSpec, organizados nas seguintes categorias:

Categoria	**Quantidade**
Lexer	8
Parser	22
Pretty Printer	6
Testes Negativos (sintático)	5
Testes de Integração (sintático)	8
TypeChecker – Programas Válidos	24
TypeChecker – Programas Inválidos	21
TypeChecker – Inferência	8
TypeChecker – Integração	8
Interpreter – Básico	9
Interpreter – Fluxo de Controle	5
Interpreter – Funções	4
Interpreter – Estruturas de Dados	5
Interpreter – Integração	4
Interpreter – Erros de Execução	5
Total	**144**

Tabela 1: Distribuição dos testes automatizados por categoria.

Exemplos da especificação. Todos os programas de exemplo fornecidos na especificação foram testados com sucesso:

Exemplo	**Modo**	**Saída**
1 – Fatorial	`-interpret`	`120`
2 – Registros e Arranjos	`-interpret`	Alice/25/1.65, Bob/30/1.8, Charlie/35/1.75
3 – Operações com Arranjos	`-interpret`	`5, 4, 3, 2, 1`
4 – Operações Matemáticas	`-interpret`	`23.02..., true, Condicao normal`
5 – Função Identidade	`-typecheck`	Passa (sem `main`)
6 – Função `map` Genérica	`-typecheck`	Passa (sem `main`)

Tabela 2: Resultados da execução dos exemplos da especificação.

Para executar todos os testes:

```
cabal test
```

4.3 Limitações

As principais limitações conhecidas da implementação atual são:

1. **Inferência de tipos limitada**: a inferência de tipos é local (baseada em expressões de inicialização), sem unificação global ao estilo Hindley-Milner. Variáveis de tipo genérico são tratadas como compatíveis com qualquer tipo, mas não há propagação completa de restrições.

2. **Ausência de geração de código**: conforme planejado, a geração de código WebAssembly será implementada na Etapa 3.

3. **Sem otimizações**: o interpretador avalia expressões por caminhamento direto na AST, sem otimizações como *constant folding* ou eliminação de código morto.

4. **Mensagens de erro sem posição precisa**: embora o analisador semântico acumule erros, as posições de linha e coluna não são propagadas desde a AST em todos os casos.

5. **Exemplos 5 e 6 apenas verificáveis**: os exemplos 5 (identidade) e 6 (map genérico) não possuem função `main`, logo não são executáveis pelo interpretador, apenas verificáveis pelo *type checker*.

5 Conclusão

Nesta segunda etapa do trabalho, implementamos com sucesso o analisador semântico e o interpretador para a linguagem SL. O analisador semântico realiza todas as verificações solicitadas na especificação: compatibilidade de tipos em operações, declaração única de identificadores no mesmo escopo, verificação de chamadas de função com número e tipos corretos de argumentos, e acesso válido a campos de registros e elementos de arranjos.

O interpretador implementa a semântica operacional de SL por meio de caminhamento na AST, suportando todas as construções da linguagem: funções (incluindo recursão), registros mutáveis, arranjos mutáveis, estruturas de controle e a função embutida `print`. A utilização de `IORef` para representar valores mutáveis mostrou-se uma decisão eficaz, permitindo a semântica de referência necessária para atribuições em arranjos e campos de registros.

Todos os exemplos fornecidos na especificação foram executados com os resultados esperados, e a suíte de 144 testes automatizados valida o comportamento correto das diversas componentes. O projeto está preparado para a terceira etapa, que envolverá a implementação da inferência de tipos completa e a geração de código WebAssembly.

6 Referências

Referências

[1] Hutton, G. (2016). *Programming in Haskell*. Cambridge University Press.

[2] Appel, A. W. (1998). *Modern Compiler Implementation in ML*. Cambridge University Press.

[3] WebAssembly Community Group. (2023). *WebAssembly Specification*.

[4] Kireev, M. (2023). *Megaparsec: Industrial-strength monadic parser combinator library*. Disponível em: `https://hackage.haskell.org/package/megaparsec`.

[5] Pierce, B. C. (2002). *Types and Programming Languages*. MIT Press.